TP1 — Passthrough & prise en main

TP

DSP temps réel sur Teensy 4.x + Audio Shield v1

Guilhem PAGES **Version 1**
guilhem.pages@univ-lemans.fr **Durée : 55 min**

? Problématique

Comment router un signal audio de l'entrée vers la sortie d'une carte Teensy en temps réel, et mesurer le coût (CPU, mémoire, latence) de cette chaîne minimale ?

Objectifs:

- Construire une chaîne audio minimale `AudioInputI2S` → `AudioOutputI2S` ;
- Configurer le codec **SGTL5000** (entrée, volume, gain d'entrée) ;
- Dimensionner `AudioMemory()` et comprendre le rôle des blocs de 128 échantillons ;
- Mesurer la charge CPU (`AudioProcessorUsage`) et l'occupation mémoire ;
- Porter un premier jugement qualitatif sur la latence.

Prérequis:

- Notions de TNS hors-ligne (échantillonnage, filtrage) ;
- Programmation C/C++ élémentaire, environnement Arduino IDE + Teensyduino installé ;
- Blink validé sur la carte (voir fiche d'installation).

Remarque

Matériel par binôme : Teensy 4.x + Audio Shield Rev D, casque sur `HP OUT`, une source audio sur `LINE IN`, câble USB *data*.

Remarque *Deux façons d'alimenter la chaîne*

Au **Jour 1**, on garde l'entrée analogique `LINE IN` : c'est elle qui permet les **mesures précises à l'oscilloscope** (latence entrée→sortie, spectre). Au **Jour 2**, on passera à l'**entrée USB** (la Teensy vue comme carte son du PC) pour écouter simplement ses effets sur de la musique, sans source ni câble externe.

1 La chaîne de flux

L'Audio Library décrit le traitement comme un **graphe** : des objets reliés par des `AudioConnection` (*patch cords*). Chaque objet reçoit et émet des **blocs de 128 échantillons** `int16`. Le moteur appelle `update()` sur chaque objet toutes les ≈ 2,9 ms (à $F_s = 44{,}1$ kHz, car $128/44100 \approx 2{,}9$ ms).

$$\texttt{AudioInputI2S} \xrightarrow{\text{L, R}} \texttt{AudioOutputI2S}$$

(contrôle du codec via `AudioControlSGTL5000`)

2 Le modèle d'exécution : `setup()` et `loop()`

Tout programme Arduino/Teensy repose sur deux fonctions imposées, appelées par le système :

- `setup()` — exécutée **une seule fois**, au démarrage ou après un reset. C'est la phase d'**initialisation** : ouverture de la liaison série (`Serial.begin`), réservation de la mémoire audio (`AudioMemory`), activation et réglage du codec (`sgtl.enable`, volume, gain), enregistrement du *callback* MIDI. Tout ce qui se règle « au début et une fois pour toutes » va ici.
- `loop()` — exécutée **en boucle infinie** juste après `setup()`, aussi vite que le processeur le permet. C'est la phase de **supervision** : lecture des faders MIDI reçus du PC, commandes clavier, affichages périodiques. Ce sont des tâches *lentes* (échelle de la milliseconde à la seconde), sans contrainte d'échantillon.

Fonction	Quand	Pour quoi
`setup()`	1 fois au démarrage	initialiser (mémoire, codec, ADC)
`loop()`	en boucle, en continu	lire le MIDI, afficher, changer les réglages
`update()`	toutes les ≈2,9 ms	**traiter l'audio** (bloc de 128 éch.)



Attention

Point crucial : le **traitement du son ne se fait NI dans `setup()` NI dans `loop()`**. Il est déclenché par le moteur audio, sur **interruption** liée au DMA du flux I2S : celui-ci appelle `update()` sur chaque objet du graphe à chaque nouveau bloc, *indépendamment* de `loop()`. Autrement dit, `loop()` et le traitement audio tournent « en parallèle ».

Deux conséquences pratiques que l'on retrouvera à chaque TP :

- `loop()` peut être lent ou occupé sans couper le son : l'audio continue de tourner en tâche de fond. On y met donc, sans crainte, les `Serial.print` et la réception MIDI (`usbMIDI.read()`).
- À l'inverse, on ne met **jamais** de `Serial.print`, de lecture MIDI ou d'attente dans `update()` : ces opérations lentes feraient exploser le budget temps réel du bloc (voir Jour 1, contraintes du temps réel).



Remarque

Un paramètre réglé dans `loop()` (ex. le gain reçu en MIDI depuis le PC) est communiqué au traitement via une simple variable membre, lue par `update()`. C'est exactement le mécanisme que l'on mettra en place au TP2 pour piloter le gain.

3 Déclaration du graphe

Activité 1

Ouvrir le squelette `jour1/TP1_passthrough/etudiant/TP1_passthrough/TP1_passthrough.ino`. Compléter les déclarations d'objets (`out`, `sgtl`) puis les deux connexions entrée → sortie (voies L et R). *Point de contrôle : le code compile même sans carte branchée.*

Squelette fourni :

```
AudioInputI2S        in;       // entrée I2S (ADC du SGTL5000)
// TODO 1 : déclarer la sortie I2S       ->  AudioOutputI2S        out;
// TODO 2 : déclarer le contrôleur codec ->  AudioControlSGTL5000  sgtl;

// -----------------------------------------------------------------------
// 2) Connexions (patch cords)
//    AudioConnection(source, canalSource, destination, canalDest)
//    canal 0 = gauche (L), canal 1 = droite (R)
// -----------------------------------------------------------------------
// TODO 3 : connecter in[0] -> out[0]  (voie gauche)
// TODO 4 : connecter in[1] -> out[1]  (voie droite)
```

4 Initialisation du codec

Activité 2

Dans `setup()`, réserver la mémoire audio puis activer et régler le SGTL5000 (entrée ligne, volume casque, gain d'entrée). Téléverser : *le son doit passer au casque, sans coupure.*



Attention

Pas de son ? Vérifier `sgtl.enable()`, la bonne entrée (`inputSelect`), le volume, et la sélection L/R du casque. Son haché / figé ⇒ `AudioMemory()` trop petit.

5 Instrumentation : CPU & mémoire

Activité 3

Ajouter dans `loop()` un affichage périodique (toutes les 1 s, **jamais** dans `update()`) de la charge

CPU et de la mémoire. Ouvrir le *Moniteur série* et relever les valeurs à vide (passthrough).

Activité 4

Réduire `AudioMemory(2)` : qu'observe-t-on ? Remonter jusqu'à un fonctionnement stable, puis laisser une marge (MemMax + 2 à +4). Consigner la valeur retenue et la justifier.

6 Latence

Activité 5

Injecter un transitoire (burst, front) issu d'un GBF sur l'entrée `LINE IN`, puis visualiser simultanément à l'oscilloscope le signal d'entrée et le signal de sortie casque. Mesurer le décalage temporel entre les deux : c'est la latence de la chaîne.

7 Livrables

1. Passthrough stéréo fonctionnel (démonstration au casque) ;
2. Relevé écrit : CPU %, CPUmax, MemMax à vide, et la valeur d'`AudioMemory()` retenue avec justification ;
3. Une phrase : combien de temps « coûte » un bloc à 44,1 kHz, et pourquoi ?



Remarque *Pour aller plus loin*

Ajouter un `AudioAnalyzePeak` sur l'entrée et afficher le niveau crête ; découvrir le pupitre MIDI (`outils/pupitre_midi.html`) en préparation du TP2.